

Cliente y autenticación

La aplicación Flutter consume la API Flask del proyecto mediante un cliente centralizado. La entrega incorpora renovación automática del token, validación por campo y sincronización de notas con almacenamiento local. Código, instrucciones y evidencia se publican en el repositorio indicado.

Configuración del cliente

Una instancia de Dio, creada en el inicio de la aplicación y compartida por las pantallas. Se eligió por sus interceptores asíncronos, tiempos de espera separados y transporte sustituible en pruebas. **Conexión: 10 s · Envío: 15 s · Recepción: 20 s.**

Desarrollo: emulador 10.0.2.2:5000/api; teléfono USB 127.0.0.1:5000/api con adb reverse. **Producción:** API_BASE_URL con HTTPS y APP_ENV=production. Toda compilación release exige HTTPS. Estos valores de compilación son públicos; no contienen secretos.

Orden de los interceptores

Orden	Interceptor y comportamiento
1	Authentication lee flutter_secure_storage en cada solicitud protegida e inyecta el encabezado Bearer. El CRUD no recibe tokens desde las pantallas.
2	Refresh renueva ante 401. Comparte una sola renovación entre solicitudes concurrentes, guarda el token antes de reenviar y marca authRetried para impedir bucles.
3	SafeRetry limita a dos reintentos, con esperas de 400 y 800 ms. Solo operaciones idempotentes; POST requiere un recibo de idempotencia en el backend.
4	SafeLog registra solo método, ruta sin consulta y estado. No registra cuerpos ni encabezados y no se incorpora en producción.

El refresh conserva el token de renovación original cuando el servidor solo devuelve un access token nuevo. Un refresh inválido termina la sesión; un fallo de conexión conserva las credenciales para recuperarse después. Cerrar sesión revoca access y refresh de forma persistente y no permite que una renovación tardía reactive la sesión.

Contrato y persistencia

Los modelos **Note** y **Session** usan `json_serializable` con comprobación de tipos. Sus archivos `.g.dart` se generan y se versionan. Las respuestas exitosas usan `data`; los listados incorporan `pagination` y el cliente recorre todas las páginas.

Modelo	JSON del servidor	Campo del cliente y regla
Note	<code>id / client_id</code>	<code>id</code> (int nullable) / <code>clientId</code> (String nullable). El ID remoto no se inventa localmente.
Note	<code>user_id / author_email</code>	<code>userId</code> / <code>authorEmail</code> ; opcionales anulables. Se conserva el autor real.
Note	<code>title / content</code>	String obligatorios; título de 3 a 100 caracteres y contenido de 1 a 5000.
Note	<code>createdAt</code>	<code>createdAt</code> ; int obligatorio, Unix en milisegundos.
Session	<code>user_id / email / role</code>	<code>userId</code> / <code>email</code> / <code>role</code> ; <code>role</code> tiene valor predeterminado <code>user</code> .
Session	<code>access_token / refresh_token</code>	<code>accessToken</code> / <code>refreshToken</code> ; String obligatorios en el almacén seguro.

Fuentes local y remota

Las pantallas usan **NotesRepository**, que coordina **ApiService** y **DBHelper**. El estado visual solicita datos al repositorio sin decidir si proceden de la red o de SQLite. Una cola de futuros serializa las operaciones y evita que el editor y la reconexión sincronicen simultáneamente.

SQLite v4 utiliza **note_cache**, **cache_metadata** y **outbox**. Distingue cuenta de caché, UUID local e ID remoto. La nota y su operación se guardan en una transacción. Se muestran antigüedad, estado sin conexión y número de pendientes. Las eliminaciones pendientes no reaparecen al refrescar.

Creaciones sin duplicados

Cada POST de creación conserva una clave única y un cuerpo inmutable. El backend guarda la nota y su recibo de idempotencia en la misma transacción. Si la respuesta se pierde, el reenvío obtiene el resultado original. Al confirmar la creación, el ID remoto se propaga a las ediciones y borrados posteriores.

Los rechazos permanentes quedan visibles para su corrección. La migración de v3 marca las operaciones antiguas para revisión porque esa versión no tenía recibos de idempotencia. Las credenciales están cifradas; el contenido local de notas no está cifrado. Al cerrar sesión se elimina la caché y cola de esa cuenta.

Errores y verificación

Familia	Traducción y recuperación
Conectividad	Mensaje sin conexión; conserva caché y cola. Un certificado inválido no se acepta ni se reintenta.
Tiempo de espera	Mensaje de espera agotada. Recuperación limitada a operaciones idempotentes.
Cliente 4xx	401 activa renovación controlada. 403 indica permisos; 409 conflicto; 422 muestra error.fields bajo cada campo.
Servidor o contrato	5xx informa indisponibilidad y permite reintento seguro. JSON incompatible se informa sin ocultarlo como lista vacía.

Seguridad comprobada

Se excluyen secretos, bases de ejecución y contraseñas del árbol de entrega. El secreto JWT se suministra desde el entorno y es obligatorio en producción. El manifiesto Android release deshabilita HTTP y backups. El backend exige HTTPS en producción, desactiva debug y usa consultas parametrizadas y autorización por cuenta.

No se registran encabezados de autorización ni cuerpos. Las sesiones revocadas siguen revocadas tras reiniciar el servidor. Los roles se consultan en la base de datos y sus cambios tienen efecto inmediato. El historial Git anterior se conserva: retirar archivos del árbol actual no borra versiones pasadas.

Pruebas reproducibles

21 pruebas del backend y 28 pruebas de Flutter aprobadas. Flutter analyze no presenta incidencias. Se comprueban renovación concurrente, 401 persistente, 422 por campo, paginación, aislamiento entre cuentas, creaciones simultáneas, respuesta perdida, edición y borrado sin conexión y recuperación de la cola.

Entorno utilizado: Flutter 3.47.4, Dart 3.13.3, Python 3.12, Java 17 y Android SDK 36. La evidencia del teléfono y el resultado de compilación se detallan en **evidence/semana13/RESULTADOS.md**; el video muestra el recorrido del taller.

Ejecutar y revisar

Backend: instalar backend/requirements.txt y ejecutar python -m backend.app. **App:** flutter pub get, dart run build_runner build y flutter run con API_BASE_URL. **Verificación:** flutter analyze, flutter test y python -m pytest -q. Para demostrar expiración, configurar ACCESS_TOKEN_SECONDS=60 antes de iniciar el backend.

El servidor integrado de Flask y la firma debug se utilizan para el taller. Un despliegue público requiere servidor WSGI con TLS, clave de firma Android propia y configuración del proxy confiable. La documentación ampliada y las referencias de Dio, serialización Flutter y JWT están en SEMANA_13.md.